SoftClient4ES

BENCHMARK REPORT

Extracting data out of Elasticsearch

SoftClient4ES over Arrow Flight SQL, measured against Trino's Elasticsearch connector — same host, same cluster, same 10-million-row index, back to back.

DATASET	SOFTCLIENT4ES	TRINO	ELASTICSEARCH
10,000,000	**0.2.5.1**	**483**	**8.18.3**
rows · 8 columns	Arrow Flight SQL sidecar	Elasticsearch connector	single node

1 Summary

This benchmark measures how efficiently each system extracts data **out of Elasticsearch** into a Python data-science client (pandas, polars, DuckDB). Both run against the same single-node Elasticsearch, the same 10-million-row index, on the same host, back to back.

> **Who this is for:** data teams pulling large result sets out of Elasticsearch into Python. It is **not** a distributed-analytics, federation, or SQL-coverage comparison — for those, Trino is the right tool and this benchmark deliberately does not exercise its strengths.

AGGREGATION PUSHDOWN

24 KB vs 1.4 GB

Data moved off the cluster for a `GROUP BY` returning 100 rows. SoftClient4ES compiles it into an Elasticsearch aggregation; Trino scans all 10M rows into its own engine.

CLIENT MEMORY FLOOR

2 GB container

Lands all 10M rows as a DataFrame inside a hard 2 GB cap. Trino's fastest client is killed there; its stock client is killed at every cap measured.

CONCURRENCY IN AN 8 GB BUDGET

5 vs 2

Five simultaneous 10M-row extractions complete — 50 million rows for +14% wall time — against two for Trino's fastest client, and none for its stock one.

10M-ROW EXTRACTION

1.48× faster

34.0 s vs 50.4 s, on 4.7× less client CPU and 4.9× less client memory — because the client consumes Arrow batches instead of building Python objects.

For work Elasticsearch can compute, SoftClient4ES pushes it into the cluster and moves almost no data off it. For large extractions it runs in far less client memory, and it is faster, because it delivers Arrow columnar batches the client consumes directly where Trino's Python client materialises rows into Python objects. Trino runs throughout as a **3-node cluster holding 1.5× our CPU and 2× our memory**, and the results also hold when the index is resharded from 1 primary shard to 5 — where the extraction gap widens rather than closes (section 7).

Where this does not apply. Below one million rows, Elasticsearch's own query language extracts faster than either engine here and can hand back Arrow directly — section 4 publishes those cells, including the ones we lose. (On the pushed-down aggregation it is the other way round: SoftClient4ES answers S3 in 0.04 s against ES|QL's 0.16 s.) ES|QL cannot return more than 1,000,000 rows, which is why the ten-million-row scenarios have no ES|QL column. **Trino is ahead on cross-index joins** — unambiguously on the join with a `GROUP BY`, within the noise on the other two — is faster on the one-million-row extraction, gains the most from shard parallelism on aggregate-heavy work, and offers capabilities — distributed execution, spill, connector breadth — this benchmark does not exercise (section 8).

> All figures in this report were measured on the **released** sidecar image `softnetwork/softclient4es8-arrow-flight-sql:0.2.5.1` (digest `sha256:90f0cf405655…`), Trino 483 and Elasticsearch 8.18.3. No number is published that did not come out of a run of the harness.

2 Environment

SoftClient4ES sidecar	`softclient4es8-arrow-flight-sql:0.2.5.1` — Arrow Flight SQL, gRPC :32010
Trino	483 (official image), Elasticsearch connector
Elasticsearch	8.18.3, single node
Index	`bench_events_10m` — 10,000,000 documents, flat mapping, 1 primary shard, 0 replicas, force-merged (916 MB)
Host	Apple M4 Max, macOS (Darwin arm64), Docker Desktop VM 10 CPU / 15.6 GiB
Resources, and the handicap they encode	**One** SoftClient4ES sidecar: 4 CPU / 4 GB. **Trino: a 3-node cluster totalling 6 CPU / 8 GB** — 1.5× the CPU and 2× the memory, in every scenario. Elasticsearch 4 CPU / 4 GB (heap 2 GB)
Sensitivity topology (§7)	`bench_events_10m_s5` — same corpus, same seed, **5 primary shards**. The engines are unchanged: only the index topology differs
ES|QL	Elasticsearch's own query language, measured as a third stack wherever it can run. Its cells were taken with `esql.query.result_truncation_max_size` raised from its 10,000 default to 1,000,000 — the product maximum; every ES|QL run records the effective value.
Runs	≥2 warm-ups, then ≥5 measured runs per cell; each run in a fresh client process. Tables show the median and the min–max spread.

Client libraries

Identical versions on both sides wherever a library is shared.

LIBRARY	VERSION	USED BY
CPython	3.12.12 (arm64)	all
pyarrow	25.0.0	SoftClient4ES (Arrow)
adbc-driver-flightsql	1.12.0	SoftClient4ES (Flight SQL)
adbc-driver-manager	1.12.0	ADBC drivers
pandas	3.0.5 (numpy 2.5.1)	both
polars	1.43.2	both (polars variant)
duckdb	1.5.5	both (S2)
trino	0.338.0	Trino (stock client)
sqlalchemy	2.0.51	Trino (`pandas.read_sql` / `pl.read_database`)
connectorx	0.4.5	Trino (fast columnar client)
ADBC Trino driver	0.5.1	Trino (ADBC client)

Measurement provenance. The single-shard extraction matrix — floors, S1, S1m, S1r, S2, S3, S4, both ES|QL wire formats, the tuned-Trino arm, the hostname-dial arm and the drift control — is **one session**: 165 measured runs, one gate, one host state. Three scenarios cannot share it because each rebuilds its own container topology, so each is its own session, run the same night on the same host and the same image: S5, S6 and the joins. Section 7 regenerates the corpus into a 5-shard index, so it is its own session too. Every table names the session it draws on; no table mixes two. *Drift:* runs are blocked by stack, so the first stack's block and the second's are separated in time. The first stack's S1 block is therefore re-run at the **end** of the session — 34.04 s at the start against 34.67 s at the end, +1.8% against a 1.4% run-to-run spread within the block. **This drift is slightly larger than the block's own noise**, so the two blocks are not interchangeable and this report says so rather than claiming a stability it did not have; it stays an order of magnitude below the 48% S1 gap it could bias, and points the wrong way for us — the later, slower block is the one Trino ran in. *Host load:* the client runs on the host while the engines run in the Docker VM, so the 1-minute load average is sampled with every run — median 4.3, maximum 9.8 against 16 logical cores, i.e. 6.2 cores uncommitted at the worst moment of the session; the harness refuses to measure above a configured ceiling rather than measuring and hoping. *Host memory pressure:* sampled with every run too, since peak client memory is the axis the two clients differ most on — macOS reported pressure level 1 (normal) on all 165 runs, with 0 MB of swap in use throughout, so no figure here was taken from a paging host.

Metrics. *Wall* is end-to-end time including connection. *Client CPU* is `time.process_time()` in the client process. *Peak client memory* is the process's peak physical footprint (`ri_lifetime_max_phys_footprint`), which is immune to the macOS memory compressor. *ES wire* is the bytes that left Elasticsearch, measured from container network counters — sampled at the Elasticsearch container itself, in every table of this report, so the figure never depends on how many containers the engine is made of. Every run asserts the exact expected row count before its timing is recorded; a run that returns the wrong number of rows is discarded, not reported.

3 Scenarios at a glance

ID	QUESTION IT ANSWERS	OUTCOME
S0	What does a single-process Python scroll client cost as a reference floor?	43.8 s to read 10M rows
S0p	And a *sliced* scroll — 5 slices, 5 processes?	**22.5 s**, for 2.9× SoftClient4ES's client CPU on S1
S1	Extract 10M rows into a client-side columnar table	**1.48× faster**, 4.7× less CPU, 4.9× less memory
S1m	Extract **1,000,000** rows — the only scale all three stacks reach	**ES|QL 0.61 s**, SoftClient4ES 3.85 s, Trino 5.18 s
S1r	Extract 10M rows into a **pandas / polars DataFrame** (what an analyst builds)	**43% faster** on far less CPU and memory
S2	Extract 10M rows and compute an aggregate in DuckDB	**1.65× faster**, 7.9× less memory
S3	`GROUP BY` returning 100 rows	**0.04 s vs 25 s**, and moves **24 KB against 1.4 GB** off the cluster
S4	Fetch 100 rows (`LIMIT 100`)	**Parity** 37 ms vs 56 ms
S5	Does the extraction fit in a constrained-memory container?	Fits **2 GB**; Trino's stock client never fits, its fastest client 3 GB
S6	How many concurrent extractions fit in an 8 GB budget?	SoftClient4ES **5** (50M rows); Trino **2** with its fastest client, **0** with its stock one
J0–J2	Cross-index JOIN landed as a DataFrame	Trino faster by 3.5% and 4.8% on two of three; SoftClient4ES 2.6% faster on the plain join and does 5–17× less client work
§7	Do the results survive a **5-shard** index, giving Trino's cluster splits to parallelise over?	Yes — S1 gap **widens to 1.52×**; client cost and pushdown unchanged; Trino's `GROUP BY` gains 4.4×

All scenarios run against one 10-million-row index (`bench_events_10m`); the JOIN scenarios also use a 1-million-row index (`bench_1m`).

4 Extraction scenarios

S0 The reference floors — a scroll client, single-threaded and sliced

A plain Python client that scrolls Elasticsearch and parses each JSON hit, counting the rows and throwing them away. Neither floor builds a usable artifact — no Arrow table, no DataFrame — which is what makes them floors rather than contenders.

FLOOR	WALL	CLIENT CPU	PEAK CLIENT MEMORY
S0 — one process, one scroll	43.8 s	14.1 s	26 MB
S0p — 5 slices, 5 processes	**22.5 s**	14.7 s	128 MB

Outcome. Scrolling single-threaded is the simplest approach, not the fastest one available to a client willing to work for it: a sliced scroll halves the wall clock for almost the same total CPU, and at 22.5 s it is **faster than either engine's S1** on this index — for **2.9× the client CPU SoftClient4ES spends on the same ten million rows in the same session** (14.7 s against 5.1 s), across five processes, returning nothing you can compute on. A reader who knows Elasticsearch would supply this comparison anyway, so it is measured rather than left out. On this single-shard index the gain comes from overlapping request round-trips rather than from shard parallelism — the summed CPU barely moves — so a multi-shard index would be expected to help this floor further, exactly as it helps Trino. That variant was not run.

S1 Extract 10M rows into a columnar client table

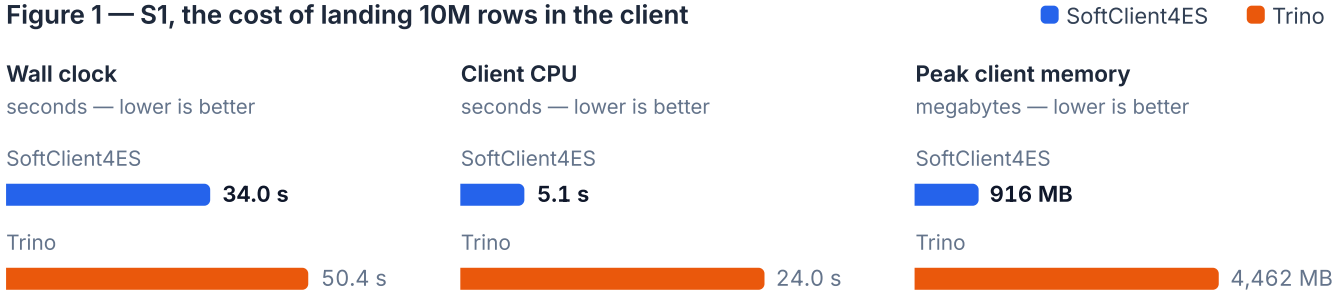
The full 10-million-row result set materialised in the client. SoftClient4ES fetches an Arrow table; Trino's stock client fetches rows.

```
# SoftClient4ES — Arrow Flight SQL (adbc_driver_flightsql)
cur.execute("SELECT id, event_ts, amount, qty, status, country, category, name "
            "FROM bench_events_10m")
table = cur.fetch_arrow_table()          # Arrow columnar batches

# Trino — stock client (trino.dbapi)
cur.execute(same_sql)
rows = cur.fetchall()                    # list of Python tuples
```

METRIC	SOFTCLIENT4ES	TRINO	RATIO
Wall median	**34.0 s (spread 1.4%)**	50.4 s (spread 1.5%)	**1.48× faster**
Client CPU	**5.1 s**	24.0 s	4.7× less
Peak client memory	**916 MB**	4,462 MB	4.9× less
ES wire	2,492 MB	2,923 MB	—
Rows / columns	10,000,000 / 8	10,000,000 / 8	✓

Figure 1 — S1, the cost of landing 10M rows in the client



Each panel has its own scale and unit; bars are comparable within a panel only. Medians of ≥5 runs after 2 warm-ups.

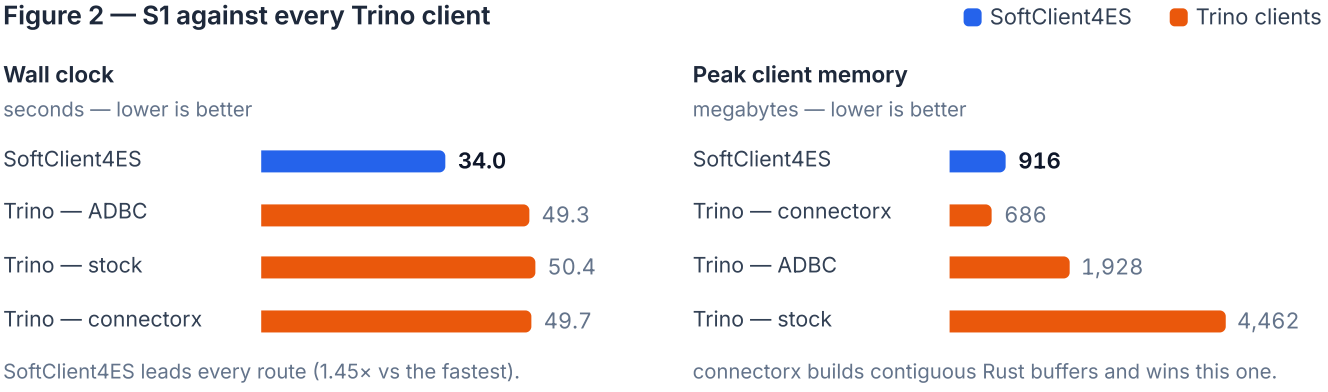
Outcome. SoftClient4ES is faster and far lighter. The reason is client representation: the client consumes Arrow batches without ever building 10 million Python objects, which is what dominates Trino's client CPU and memory.

Fairness — measured against Trino's fastest clients

Trino's stock client is not its only option. **connectorx** (a Rust engine that parses Trino's result pages straight into columnar buffers) and the **ADBC Trino driver** both return an Arrow table. Measured on the same query:

ROUTE	WALL	CLIENT CPU	PEAK MEMORY
SoftClient4ES — Arrow Flight SQL	**34.0 s**	5.1 s	916 MB
Trino — ADBC driver	49.3 s	18.7 s	1,928 MB
Trino — connectorx	49.7 s	11.0 s	**686 MB**
Trino — stock client	50.4 s	24.0 s	4,462 MB

Figure 2 — S1 against every Trino client



SoftClient4ES leads every route (1.45× vs the fastest).

connectorx builds contiguous Rust buffers and wins this one.

The counter-evidence is part of the result. Panels have independent scales.

SoftClient4ES keeps the wall-clock lead against every Trino client (1.45× vs the fastest). On client cost the picture is honest: connectorx, building contiguous buffers in Rust, reaches a lower peak memory (686 MB) than our chunked Arrow batches. **The wall-clock advantage comes from the server and the wire, not from any client library.** Note that 686 MB is a bare Arrow *table*: when the same workflow materialises the whole result as a DataFrame (scenario S5), connectorx needs 2.9 GB where SoftClient4ES needs 1.5 GB — so this memory advantage does not carry to the DataFrame an analyst actually keeps.

S1r Extract 10M rows into a DataFrame

The artifact a data scientist actually builds. Each side uses its most idiomatic route to a `pandas.DataFrame`, and separately to a `polars.DataFrame`.

```
# SoftClient4ES
df = cur.fetch_arrow_table().to_pandas()        # pandas
df = pl.from_arrow(cur.fetch_arrow_table())     # polars

# Trino
df = pandas.read_sql(sql, sqlalchemy_conn)      # pandas, via the trino SQLAlchemy dialect
df = pl.read_database(sql, sqlalchemy_conn)     # polars
```

To a pandas DataFrame

ROUTE	WALL	CLIENT CPU	PEAK MEMORY
SoftClient4ES (`to_pandas()`)	**34.7 s**	5.2 s	1,473 MB
Trino — connectorx	49.6 s	11.7 s	2,823 MB
Trino — ADBC driver	50.4 s	19.4 s	2,553 MB
Trino — SQLAlchemy (stock)	60.9 s	35.3 s	7,966 MB

Against Trino's stock route that is **43% faster on 6.8× less CPU and 5.4× less memory**; against its fastest route, 30% faster. With Arrow-backed pandas dtypes (`types_mapper=pd.ArrowDtype`), SoftClient4ES needs only **916 MB** (34.3 s), versus 7,800 MB for the stock Trino route.

To a polars DataFrame

ROUTE	WALL	CLIENT CPU	PEAK MEMORY
SoftClient4ES (`pl.from_arrow`)	**35.1 s**	14.6 s	2,480 MB
Trino — connectorx	50.3 s	11.8 s	**2,238 MB**
Trino — ADBC driver	50.8 s	28.4 s	3,558 MB
Trino — SQLAlchemy (stock)	61.2 s	35.2 s	11,046 MB

Outcome. SoftClient4ES lands a DataFrame faster on every route and destination — 40% faster than Trino's stock route on polars, 43% on pandas, and 30% faster than its fastest route on either. On the polars destination both sides re-encode Arrow strings into polars' native format, which raises our client CPU; connectorx does the same re-encoding in Rust and comes in slightly lower on CPU and memory. The durable advantage, again, is wall-clock, held on every route.

S2 Extract 10M rows and aggregate in DuckDB

The same fetch, landed in DuckDB and aggregated. SoftClient4ES registers the Arrow table zero-copy; Trino builds a DataFrame from rows and registers that.

```
# SoftClient4ES
con.register("events", cur.fetch_arrow_table())          # zero-copy
con.execute("SELECT category, AVG(amount) FROM events GROUP BY category")

# Trino
df = pandas.DataFrame(cur.fetchall(), columns=cols)
con.register("events", df)
con.execute(same_agg)
```

METRIC	SOFTCLIENT4ES	TRINO	RATIO
Wall median	**34.8 s**	57.5 s	**1.65× faster**
Client CPU	**6.1 s**	32.3 s	5.3× less
Peak client memory	**948 MB**	7,506 MB	7.9× less

Outcome. The zero-copy Arrow hand-off to DuckDB is the largest memory advantage in the benchmark (7.9×): the columnar result is scanned in place instead of being copied through Python objects.

S3 `GROUP BY` returning 100 rows

`SELECT category, COUNT(*), AVG(amount) FROM bench_events_10m GROUP BY category`, returning 100 groups.

METRIC	SOFTCLIENT4ES	TRINO
Wall median	**0.04 s**	25.1 s
ES wire	**24 KB**	1,394 MB
Rows	100	100

The same answer, not merely the same row count. A `terms` aggregation is approximate when its bucket size is too small, so "100 groups" is not by itself evidence that the pushed-down result matches a full scan. Every session gates on the values: all 100 `(category, COUNT, AVG(amount))` triples are compared across the three stacks — identical counts, averages equal to within 1e-9 — and Elasticsearch reports `doc_count_error_upper_bound = 0` and `sum_other_doc_count = 0`, its own statement that no bucket is missing and no document fell outside the returned buckets.

Outcome. SoftClient4ES compiles the `GROUP BY` into an Elasticsearch `terms` aggregation: the cluster computes the 100 groups and returns only those 100 rows — 24 KB of aggregation response against the 1,394 MB Trino reads to compute the same answer, a factor of 57,000. Trino's Elasticsearch connector performs predicate push-down only (per its documentation), so it scans all 10M rows into Trino and aggregates there. **This is the clearest architectural difference in the benchmark** — for work Elasticsearch can do, SoftClient4ES does not move the data at all.

S1m Extract 1,000,000 rows — the only scale all three stacks reach

Elasticsearch's own query language cannot return more than 1,000,000 rows, so this is the largest result set on which SoftClient4ES, Trino and ES|QL can be compared like for like. Same eight columns, same index, `LIMIT 1000000` on every stack.

ROUTE	WALL	CLIENT CPU	PEAK CLIENT MEMORY	BYTES OFF THE CLUSTER
ES|QL, `format=arrow`	**0.61 s**	0.09 s	166 MB	72 MB
ES|QL, `format=json`	1.29 s	0.55 s	667 MB	86 MB
SoftClient4ES, Arrow Flight SQL	**3.85 s**	**0.22 s**	174 MB	247 MB
Trino, stock client	5.18 s	2.24 s	473 MB	291 MB

Outcome — ES|QL wins this scale; between the two SQL engines we are 1.35× faster. ES|QL remains an order of magnitude faster than anything else measured here, and that is the honest headline of this cell. Against Trino, SoftClient4ES lands the same million rows in 3.85 s against 5.18 s, on **10× less client CPU** (0.22 s against 2.24 s) and 2.7× less client memory. The reason ES|QL is fast is not the wire: it reads `doc_values`, columnar on disk, while both engines read `_source` and pay a JSON parse per document. **The wire explains the ordering.** For the same million rows SoftClient4ES moves **247 bytes per row off the cluster against Trino's 291**, at the same cost per byte as its own ten-million-row run, which moves **249**. The million-row figure is the ten-million-row behaviour at one tenth the scale, not a different regime.

ES|QL Elasticsearch's own query language, and where it stops

A benchmark of two SQL engines over Elasticsearch that never measures what Elasticsearch itself can do invites an obvious question. So it is measured, over both of its wire formats: the row-shaped `format=json` every ES|QL client speaks, and `format=arrow`, which returns an Apache Arrow IPC stream.

SCENARIO	ES|QL (ARROW)	ES|QL (JSON)	SOFTCLIENT4ES	TRINO
S1m — 1,000,000 rows	**0.61 s**	1.29 s	3.85 s	5.18 s
S3 — `GROUP BY`, 100 rows	0.22 s	0.16 s	**0.04 s**	25.05 s
S4 — `LIMIT 100`	0.006 s	**0.005 s**	0.037 s	0.056 s
S1, S2, S5, S6 — 10,000,000 rows	cannot run	cannot run	✓	✓

All four columns are the same session as the sections above.

Where it wins. Below its ceiling ES|QL is the fastest way to get rows out of Elasticsearch that we measured: on a 100-row fetch its whole round trip (4 ms) costs less than SoftClient4ES's connection handshake alone (13.0 ms). Trino's handshake is cheaper still (1.7 ms); its 56 ms total is spent elsewhere. **Where it does not.** On the pushed-down aggregation SoftClient4ES is 4.1× faster while both move kilobytes rather than gigabytes off the cluster — 24 KB for us, 42 KB for ES|QL. The work is identical; the difference is what the result travels over. **Why it is absent from four scenarios.** `esql.query.result_truncation_max_size` defaults to 10,000 rows and is declared with a hard maximum of 1,000,000; Elasticsearch refuses a higher value outright. Ask for ten million rows with the setting at its maximum and the response is 1,000,000 rows, HTTP 200, and no `Warning` header — a truncated answer that looks like a complete one, recorded as `esql-truncation-probe.json` in the session directory. Not a licence gate: the cluster measured here runs a `basic` licence. For result sets under a million rows, ES|QL is an excellent answer and this benchmark is not the argument for anything else. **The second boundary is the join.** `LOOKUP JOIN` does not join two fact indices: it resolves only against an index in `lookup` mode, and such an index is **restricted to a single shard** — a dimension table. Asked for this benchmark's cross-index join it answers `400 — invalid [bench_1m] resolution in lookup mode to an index in [standard] mode`, so section 6's joins have no ES|QL column either: not that it is slower, but that the shape is outside what the feature accepts.

S4 Fetch 100 rows (`LIMIT 100`)

The control: when the result is small, the client representation stops mattering.

METRIC	SOFTCLIENT4ES	TRINO
Wall median	0.037 s	0.056 s

Outcome. Near parity, as expected. The extraction advantage only appears at scale, or when work can be pushed into Elasticsearch.

What the 18 ms actually is, and a control that removes it. At this size the whole scenario is connection setup: SoftClient4ES's ADBC Flight SQL handshake takes 13.0 ms of the 38, Trino's 1.6 ms of the 56. Both clients dial an IP literal here, deliberately — because when SoftClient4ES is pointed at `localhost` instead, its connect cost rises by ~16 ms and the S4 median moves to **0.053 s**, which at these spreads (~20%) is no longer distinguishable from Trino's 0.056 s. The margin on this control is therefore worth about as much as a name lookup, and it disappears if the client resolves one. The cause is client-side and measured: a four-layer probe separates a bare TCP connect (0.09 ms) from the Flight C++ layer (1.6 ms), the Go ADBC layer dialling an IP (3.0 ms) and the same layer dialling a name (17.7 ms). grpc-go performs its own resolution rather than using the OS resolver, which is where the ~15 ms goes — the host's own resolver answers the same name in 0.20 ms. No other scenario is affected — 15 ms is invisible against 34 seconds — but on a 100-row fetch it is the whole result.

Correctness gate — run before any of the timings above. Row counts alone do not establish that a pushed-down aggregation returns the same answer as a full scan, so each session first runs a data-equivalence gate, and a session whose gate does not pass is not measured at all. **All three stacks take part in both halves.** On the push-down itself — every one of S3's 100 `(category, cnt, AVG(amount))` triples, key by key and value by value: they agreed exactly on the counts and to within 1e-9 on the averages. And on `COUNT`, `SUM(id)`, `SUM(qty)` and `SUM(amount)` over the whole index — the check that the extraction paths see the same 10,000,000 documents, not merely the same number of them: all three agreed on all four. Elasticsearch's own error bounds for the `terms` aggregation were `doc_count_error_upper_bound = 0` and `sum_other_doc_count = 0` — the aggregation is exact at this shard count and bucket size, not merely close.

5 Constrained memory and concurrency

S5 Does the extraction fit in a small container?

The client runs inside a container with a hard memory cap (`docker run --memory`, swap pinned equal so the kernel kills rather than pages). The question is binary: does landing 10M rows as a DataFrame complete, or is the process killed?

Whole result set as one DataFrame

CONTAINER CAP	SOFTCLIENT4ES	TRINO (STOCK)	TRINO (CONNECTORX, FASTEST)
8 GB	completes 34.2 s · 1,525 MB	OOM-killed	completes 50.4 s · 2,910 MB
6 GB	completes 35.9 s · 1,527 MB	OOM-killed	completes 49.5 s · 2,902 MB
4 GB	completes 35.3 s · 1,527 MB	OOM-killed	completes 48.9 s · 2,907 MB
3 GB	completes 35.2 s · 1,538 MB	OOM-killed	completes 48.5 s · 2,912 MB
2 GB	completes **35.3 s · 1,527 MB**	OOM-killed	OOM-killed

The whole table comes from one sweep per client. "OOM-killed" is the kernel killing the client — exit 137 — in every cell above, verified per cell.

Streaming, where neither side holds the whole result — SoftClient4ES via `fetch_record_batch()`, Trino via `pandas.read_sql(chunksize=…)`. This is the workflow a competent Trino user reaches for first, so it is a table rather than a footnote:

CONTAINER CAP	SOFTCLIENT4ES	TRINO (STOCK)
8 GB	completes 36.2 s · 146 MB	completes 58.7 s · 303 MB
6 GB	completes 36.1 s · 147 MB	completes 58.5 s · 305 MB
4 GB	completes 37.1 s · 146 MB	completes 58.4 s · 305 MB
3 GB	completes 36.1 s · 146 MB	completes 58.4 s · 302 MB
2 GB	completes **36.2 s · 147 MB**	completes **58.1 s · 304 MB**

Both complete at every cap. SoftClient4ES is **1.58–1.62× faster on 2.1× less memory** — a smaller and more representative difference than the binary one above, and the one that applies whenever the workflow does not need the whole result set in memory at once.

> **Outcome.** Landing the whole 10M-row DataFrame, SoftClient4ES fits in a **2 GB** container. Trino's stock client is killed at every cap measured, 8 GB included; its fastest client (connectorx) fits at 3 GB but is killed at 2 GB — the cap where SoftClient4ES still completes. Both requirements are independent of the cap (peak memory moves by under 1% across a 4× range), so they reflect the data, not memory thrashing. The binary framing is the harder claim, and it answers "can I materialise this result in a small container", not "can this pipeline run at all" — read it next to the streaming table above, where both engines run everywhere and the difference narrows to about 1.6×.

S6 Concurrent extractions in a fixed memory budget

An 8 GB total client budget, split evenly across N clients launched simultaneously, each extracting 10M rows. How many complete with the correct row count? Measured against **both** of Trino's clients — the stock one it ships and connectorx, the fastest it has, which S1 already grants it.

ENGINE	N	CAP EACH	COMPLETED	WALL
SoftClient4ES	1	8,192 MB	1 / 1	34.9 s
SoftClient4ES	2	4,096 MB	2 / 2	35.6 s
SoftClient4ES	3	2,730 MB	3 / 3	36.1 s
SoftClient4ES	4	2,048 MB	4 / 4	38.0 s
SoftClient4ES	**5**	**1,638 MB**	**5 / 5**	**39.9 s**
Trino — connectorx	1	8,192 MB	1 / 1	49.8 s
Trino — connectorx	**2**	**4,096 MB**	**2 / 2**	**51.0 s**
Trino — connectorx	3	2,730 MB	0 / 3	killed
Trino — stock client	1	8,192 MB	0 / 1	killed

> **Outcome.** In an 8 GB budget, SoftClient4ES completes **five** concurrent extractions — 50 million rows for 5× the work at **+14% wall time** — against **two** for Trino's fastest client and none for its stock one. The ratio is 2.5×, not the five-versus-nothing the stock client alone would suggest, and it follows directly from what each client holds per extraction. This measures *client-side* capacity; it does not measure server-side concurrency, which is a separate question this benchmark does not address.

6 Cross-index JOIN

A join between a 1M-row index (`bench_1m`) and the 10M-row index, landed as a pandas DataFrame on both sides. Both engines execute the join server-side (SoftClient4ES in an embedded DuckDB, Trino in its own engine).

```
-- J0  plain join                    (1,000,000 rows out)
SELECT a.id, b.amount FROM bench_1m a JOIN bench_events_10m b ON a.id = b.id
-- J1  + predicate on the large leg  (125,361 rows out)
SELECT a.id, b.amount FROM bench_1m a JOIN bench_events_10m b ON a.id = b.id WHERE b.status =
'paid'
-- J2  + GROUP BY                     (100 rows out)
SELECT b.category, COUNT(*), AVG(b.amount) FROM bench_1m a JOIN bench_events_10m b ON a.id =
b.id GROUP BY b.category
```

SCENARIO	SOFTCLIENT4ES WALL	TRINO WALL	CLIENT CPU (SC4ES / TRINO)
J0 — plain join	**25.95 s** (2.6% faster)	26.63 s	0.07 s / 1.20 s
J1 — + `WHERE`	10.50 s	**10.13 s** (3.5% faster)	0.03 s / 0.20 s
J2 — + `GROUP BY`	28.74 s	**27.36 s** (4.8% faster)	0.02 s / 0.09 s

Medians of 5 runs, as everywhere else. These are small differences, so the spread belongs next to them — and each engine's 5 runs are compared against the other's 5, 25 pairings per scenario, so the ordering does not rest on the medians alone:

SCENARIO	SOFTCLIENT4ES MIN–MAX	TRINO MIN–MAX	GAP	RUNS WON, OF THE 25 PAIRINGS
J0	25.17–26.12 s (3.6%)	26.04–26.85 s (3.0%)	2.6%	**SC4ES 24 / 25**
J1	10.48–10.52 s (0.4%)	10.12–10.17 s (0.5%)	3.5%	**Trino 25 / 25**
J2	28.65–28.87 s (0.8%)	26.64–27.46 s (3.0%)	4.8%	**Trino 25 / 25**

Outcome. Trino is faster on the two joins that add work to the join itself — J1 by 3.5% and J2 by 4.8%, each winning all 25 pairings — while **SoftClient4ES is 2.6% faster on the plain join**, winning 24 of 25. The comparison was reached on a single-shard index that does not exercise its split parallelism. SoftClient4ES does **5–17×** less client-side work in every case. The marginal cost of adding a `GROUP BY` to the join is higher for SoftClient4ES (+2.8 s vs +0.7 s), so on join-side aggregation Trino is the more efficient engine, and that is the clearest of the three results.

7 Index topology sensitivity

Everything above was measured on a single-shard index. Trino's Elasticsearch connector creates one split per shard, so a single-shard index gives its cluster a single reader however many workers it has — which invites the fair question of whether these results survive an index that lets it parallelise.

They do, and the extraction gap widens.

> **Setup.** Same host, same engines, same corpus regenerated from the same seed into a **5-shard** index, one segment per shard. **Only the shard count changes**: the engines, their resources and the 1.5×-CPU handicap in Trino's favour are the ones stated in section 2, unchanged. Medians of 5 runs after 2 warm-ups. The parallelism under test was verified rather than assumed: a live probe of `system.runtime.tasks` during the scan recorded **5 splits, 3 on one worker and 2 on the other, none on the coordinator**.

METRIC	SOFTCLIENT4ES 1 SHARD	SOFTCLIENT4ES 5 SHARDS	TRINO 1 SHARD	TRINO 5 SHARDS
S1 wall	34.0 s	**30.1 s**	50.4 s	45.7 s
S1 client CPU	5.1 s	**4.6 s**	24.0 s	24.2 s
S1 peak client memory	916 MB	**916 MB**	4,462 MB	4,464 MB
S1 ES wire	2,492 MB	2,551 MB	2,923 MB	2,949 MB
S3 wall (`GROUP BY`, 100 rows)	0.04 s	**0.04 s**	25.1 s	5.7 s
S3 ES wire	24 KB	**24 KB**	1,394 MB	1,419 MB
S5 — 10M rows in a 2 GB container	completes	completes	killed	killed

Extraction. Both engines benefit from the extra shards, and SoftClient4ES benefits slightly more: 12% faster against Trino's 9%. The S1 ratio **widens from 1.48× to 1.52×**, on a topology chosen to favour Trino and with Trino holding half again our CPU. The widening is small; what the section establishes is that the gap does not close.

Client cost is a property of the protocol, not the topology. Peak client memory moves by **0.02% on our side and 0.01% on Trino's**; client CPU is flat to slightly lower for both. Sharding the index and adding Trino nodes does not change what the client pays, because the client is one process consuming one wire format however large the cluster is. The constrained-container result is unchanged with it: 10M rows still land as a DataFrame in 2 GB on one side and are still OOM-killed on the other.

Pushdown is architectural. The `GROUP BY` still moves **24 KB off the cluster against 1,419 MB** — the 100-row answer itself, and a factor of 58,000. Five shards and two extra Trino nodes do not change it, because the connector pushes no aggregation down: it reads all 10 million rows whatever the topology, and merely reads them faster.

Where Trino gains most

Its aggregation wall-clock improves **4.4×** (25.1 s → 5.7 s), by far the largest single improvement measured in this benchmark for either engine. Scan parallelism is exactly what a multi-shard index gives Trino, and on aggregate-heavy work it converts directly into wall-clock.

Both ES-wire columns in this section are measured at the Elasticsearch container itself — bytes that left the cluster — which matters here because with a three-node Trino the coordinator's own counter sees only a fraction of the scan.

8 Where Trino is stronger

A fair benchmark names the other system's strengths.

- **Up to one million rows, Elasticsearch itself extracts faster than either engine** — ES|QL returns 1M rows in 0.61 s over Arrow against SoftClient4ES's 3.85 s and Trino's 5.18 s, and a 100-row fetch in 5 ms against 37 ms (section 4). It does not beat us on the pushed-down aggregation, where we are 4.1× faster, and its 1,000,000-row ceiling is why it is absent from the rest of this report. This is a loss to Elasticsearch, not to Trino.
- **Aggregation over a sharded index** — given shards to parallelise over, Trino's `GROUP BY` wall-clock improves **4.4×** (25.1 s → 5.7 s on a 5-shard index), the largest single gain either engine showed in this benchmark. It still moves 1.4 GB off the cluster to get there.
- **Cross-index joins** — Trino is faster on the two joins that add work to the join itself (J1 by 3.5%, J2 by 4.8%, both with non-overlapping ranges) and is markedly more efficient at aggregating over a join: adding the `GROUP BY` costs it +0.7 s against our +2.8 s. We are 2.6% faster on the plain join.
- **A 100-row fetch is a tie once a name lookup is involved** — our 19 ms margin on S4 is worth about one hostname resolution in our gRPC client, and vanishes when the client dials `localhost` instead of an IP (section 4). grpc-go resolves names itself in ~15 ms where the host's own resolver answers in 0.20 ms.
- **Its page size is not tuned in these results, and tuning it helps a little** — raising `elasticsearch.scroll-size` from its default 1000 to 5000 improves Trino's S1 wall clock from 50.4 s to **48.6 s** (3.6%). The headline tables pair product default against product default; this is what the other setting is worth on this corpus.
- **A lower-memory Arrow client exists** — via connectorx, Trino can land an Arrow *table* in less client memory than SoftClient4ES (686 MB vs 916 MB on S1), and with less CPU and memory on the polars destination. (This does not extend to the full DataFrame, where SoftClient4ES uses less — S5.)
- **Distributed execution, spill-to-disk and fault tolerance** — Trino scales a single query across a cluster far larger than the 3 nodes it runs on here, and spills to disk; this benchmark does not exercise any of it.
- **Connector breadth** — Trino federates 40+ data sources; SoftClient4ES is Elasticsearch-focused.
- **Licensing** — Trino is Apache-2.0 throughout; SoftClient4ES gates result-set size by licence tier (see Reproduction).

9 Reproduction

Prerequisites: Docker (VM ≥ 12 GB RAM / ≥ 12 CPUs), a native CPython 3.12, and ≥ 32 GB host RAM — the client runs on the host, and Trino's stock client materialises 10M rows into Python objects.

```
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt

# Bring up Elasticsearch and load the data (deterministic; ~3 minutes)
docker compose up -d elasticsearch
.venv/bin/python generator/generate_and_load.py

# Bring up both engines
docker compose up -d flight-sql trino

# Extraction matrix S0–S4 (medians of 5 runs after 2 warm-ups)
.venv/bin/python runners/orchestrate.py --stop-idle-engine

# Constrained-memory (S5) and concurrency (S6)
.venv/bin/python runners/orchestrate_capped.py
.venv/bin/python runners/orchestrate_concurrent.py --budget 8

# Cross-index JOIN (J0–J2)
.venv/bin/python runners/orchestrate_join.py
```

Results are written under `results/<session>/`, one JSON per run, with the environment and the sidecar image digest captured automatically.

> **Licence.** SoftClient4ES enforces a result-set quota by licence tier (Community 10,000 / Pro 1,000,000 / Enterprise unlimited), so the full 10M rows (S1/S2) run under an Enterprise-tier licence. **The licence lifts a row quota only — it does not change the batch size, the serialization, or the data path.** The runners assert exactly 10,000,000 rows, so a quota-capped run aborts rather than being reported — a truncated result can never be published by accident. With no licence the harness runs on the Community tier, which reproduces every scenario's shape at reduced (`--limit`) scale.

A Appendix — Methodology

What this benchmark measures — and what it does not

The data path under test:

```
Elasticsearch → row-serialized documents out of the cluster (the _search / _sql path
                both engines read; ES|QL's columnar Arrow output is a third path,
                measured separately and capped at 1,000,000 rows)
             → SoftClient4ES: converted to Arrow ONCE at the sidecar, streamed as
               Arrow batches, consumed by the client with no further deserialization
             → Trino: parsed into Trino's columnar pages, re-serialized to Trino's
               row-oriented client protocol, re-parsed value-by-value in the client
```

Every client pays an Elasticsearch serialization leg. Nothing leaves the cluster unserialized, and this benchmark never claims to avoid that step. For the SQL and JDBC path it is about, that serialization is **row-shaped**: `_search` returns JSON, and the SQL endpoint offers CSV, TSV, YAML and the binary CBOR and Smile — every one of them a row-serialized document encoding.

One Elasticsearch format is columnar, and it is measured here rather than glossed over: ES|QL's `format=arrow` returns an Apache Arrow IPC stream (verified on Elasticsearch 8.18.3), and below its ceiling it *extracts* faster than either engine in this benchmark — though not on the pushed-down aggregation, where SoftClient4ES is 5× faster. That ceiling is why it appears in three scenarios and not in the rest: `esql.query.result_truncation_max_size` defaults to 10,000 rows and is declared with a hard maximum of 1,000,000, so ES|QL cannot return a ten-million-row result at all — and above the configured ceiling it truncates with HTTP 200 and no `Warning` header.

What this benchmark measures is what happens *after* the serialization leg, on the path a SQL client takes: SoftClient4ES serializes to Arrow once and the client consumes that columnar buffer directly, while Trino re-serializes to a row protocol the client must parse again.

So this benchmark measures **large result-set extraction and client-side consumption** — wall-clock, client CPU, peak client memory — which is where the client representation dominates. It does **not** measure Elasticsearch-side query execution (I/O, scoring), where the wire format is irrelevant.

Fairness rules

- **Same host, same session, same index.** The whole single-shard matrix is one session (one gate, one host state, 165 measured runs); S5, S6 and the joins each rebuild their own container topology, so each is its own session, run the same night on the same host and image. Every stack measured in a scenario — SoftClient4ES, Trino, and ES|QL where it can run — reads `bench_events_10m` back to back.
- **Container limits, and the handicap they encode:** Elasticsearch and the sidecar each get 4 CPU / 4 GB; **Trino gets more, deliberately** — a 3-node cluster totalling 6 CPU / 8 GB, in every scenario. Elasticsearch heap is pinned at 2 GB; Trino's official image auto-sizes its heap from container memory.
- **Page-size symmetry:** the sidecar's `ARROW_BATCH_SIZE=1000` equals Trino's `elasticsearch.scroll-size=1000`. Both are product defaults, pinned explicitly so the setting can be checked.
- **Authentication disabled on both paths**, so authentication cost is zero and identical on both sides.
- **Both clients are used the way their projects document them.** SoftClient4ES is driven with the standard `adbc_driver_flightsql` Arrow Flight SQL driver; Trino with its own `trino` Python package, and — for the fairness comparison in S1/S1r — with its fastest available clients (connectorx and the ADBC Trino driver). No SoftClient4ES-specific client code is used.

- **Trino uses its default type mapping.** `legacy_primitive_types` would return unparsed strings, deferring the parse rather than avoiding it, and is not used.
- **Warm cache, fresh process.** Each scenario's warm-ups run immediately before its measured runs, so both systems measure against a warm Elasticsearch page cache. Every measured run is a fresh OS process, so peak memory cannot leak between runs. Because SoftClient4ES runs first in each scenario, Trino meets a slightly warmer cache — an effect in Trino's favour.
- **Correctness before timing.** Each run asserts the exact expected row count; a run that returns the wrong count is discarded, never reported. Python is never run with `-O`, so the asserts always execute.
- The exact configuration lives alongside the harness, and the data generator is deterministic (fixed seed).

Metrics

- **Wall-clock:** `time.perf_counter()` around connect → materialized result. Connection setup is included because it is time the user waits. **Every stack dials an IP literal**, so no arm resolves a name. Trino's Python clients resolve through the OS resolver; the Go ADBC driver uses grpc-go's own, which ignores `/etc/hosts`. A four-layer connect probe separates the cost: bare TCP 0.09 ms, the Flight C++ layer 1.6 ms, the Go ADBC layer dialling an IP 3.0 ms, and **the same layer dialling a name 17.7 ms** — a ≈15 ms resolver tax that belongs to the driver, not the protocol, with a 5 s DNS timeout behind it (softclient4es-arrow#151). The scale decides whether that matters: 0.04% of a ten-million-row extraction, but the whole result on the 100-row control. Both dials are measured and both published: by IP, S4 is 0.037 s against Trino's 0.056 s; by name it is 0.056 s, exactly Trino's. The name lookup does not invert that control, it **erases** it. Published as a deployment note — with the Go driver, dial an address or reuse the connection.
- **Client CPU:** `time.process_time()` for the client process — the CPU spent turning the wire format into usable values.
- **Peak client memory:** the process's peak physical footprint. On macOS this is the kernel's `ri_lifetime_max_phys_footprint`, which includes compressed pages and is therefore immune to the macOS memory compressor (plain RSS under-reports under memory pressure). On Linux the harness falls back to `ru_maxrss`.
- **ES wire bytes:** read from the container network counters, so a wire-volume difference is measured, not asserted. The counter is read **at the Elasticsearch container** in every table of this report, section 7 included, so the figure cannot depend on how many containers the engine is made of.
- **Server-side cost is not measured.** Neither the Elasticsearch container, the sidecar nor Trino has its CPU or memory recorded per run. Every resource figure in this report is a **client** figure — an aggregation push-down is credited with the bytes it does not move, not with the cluster CPU it consumes, and the sidecar's own JVM cost sits outside the comparison exactly as Trino's does.

Interpretation guardrails

- **S1 / S1r / S2** differences are attributable to the wire format and client-side representation.
- **S3** differences are attributable to **aggregation pushdown**, not the wire format: SoftClient4ES compiles the `GROUP BY` into an Elasticsearch aggregation, while Trino's connector performs predicate pushdown only and scans all rows. S3 is never a wire-format result.
- **S4** is the control: with a small result the wire format stops mattering, and near-parity there is a sign the benchmark is honest, not a weakness.
- **Where Trino is stronger** is published alongside the results (section 8).

Known limitations

- **Single node throughout.** This does not exercise Trino's distributed execution, spill-to-disk or fault tolerance — real strengths of Trino that an extraction benchmark cannot show.
- **One flat mapping, no nested fields or arrays.** This avoids Trino's documented gaps in those types, so the connector is not handicapped; it also means the benchmark says nothing about SQL coverage, only about extraction efficiency.
- **One column shape: eight columns over five types** (`long`, `date`, `double`, `integer`, `keyword`). The client-side cost of a wire format is a function of column count and type mix, and only one point on that curve is measured. Nothing here establishes how the gap moves for two columns or eighty.
- **Only the client's half of the cost is measured.** A reading of these results as "total system cost" is not supported by anything in this harness.
- **Warm cache only, by construction.** No cold-cache arm exists, so nothing here describes a first query after a restart.
- **The constrained-memory and concurrency scenarios run their client inside a Linux container**, while the rest run it natively on macOS. Wall-clock is comparable within each group and only indicative across them, and the peak-memory metric changes with the platform.
- **The ten-million-row scenarios are not reproducible on the Community tier.** They require a licence whose result quota exceeds 10M. A third party can reproduce every scenario's shape at reduced scale, and S1m/S3/S4 exactly, but cannot independently re-run the headline extraction without one.
- **Single primary shard in the main results — and that setup favours SoftClient4ES on wall-clock.** Trino's connector creates one split per shard, so a single-shard index gives it one reader and its scan parallelism is not exercised. Note what bounds this: the *shard* count, not the node count — additional Trino workers cannot split a single reader.

  This is no longer left as a caveat: **section 7 publishes the measured sensitivity run** — the same corpus regenerated from the same seed into a 5-shard index, read by a real 5-shard index — the engines unchanged, Trino the same 3-node cluster on 6 CPU / 8 GB against our 4 CPU / 4 GB — with the split distribution captured live from `system.runtime.tasks`. Both engines get faster; the S1 gap widens from **1.48× to 1.52×**; client CPU, peak client memory (±0.04%) and the 2 GB container threshold do not move; the `GROUP BY` still moves 24 KB against 1.4 GB. Trino's own largest gain in the whole benchmark appears there too — its aggregation wall-clock improves **4.4×** — and is published as such.

  The distinction the run establishes: **wall-clock is topology-sensitive; client cost and pushdown are not.** The client is one process consuming one wire format however large the cluster, and the connector pushes no aggregation down at any shard count. That is now measured rather than argued.
- **Trino's spooling protocol was not benchmarked separately.** That variant is superseded by the S1/S1r fairness comparison against connectorx and the ADBC Trino driver, which measure Trino's fastest *columnar* clients directly — a stronger fairness position than the spooling protocol alone.

B Appendix — Issues found and fixed while benchmarking

Building this benchmark surfaced several issues in SoftClient4ES. All were fixed, and the results in this report were measured on the released `0.2.5.1` build that contains the fixes.

Extraction performance

- **Deep-paging sort regression (Elasticsearch 8+).** For a query with no `ORDER BY`, the streaming pager injected `_shard_doc` as the primary sort. From Elasticsearch 8 / Lucene 9 onward this defeats the

document-id skip optimisation, so each page re-scanned the whole index instead of seeking to the next position — about **60 ms/page instead of 8 ms/page** on the 10M-row index, an effect that grows with index size. The fix uses `_doc` as the sort under a point-in-time context (which Elasticsearch turns into a total order via an automatic tiebreaker), restoring efficient paging while keeping results complete. A multi-shard completeness test guards against regression.

- **Schema probe re-ran bounded statements.** To advertise a result's schema the Flight endpoint executed the statement itself; when the statement already carried a `LIMIT` it was executed in full, so a bounded extraction read every row off Elasticsearch **twice**. The probe now reads at most 100 rows and never more than the statement asked for, and aggregation-shaped statements — where the limit is the bucket count, not a row count — are left untouched.
- **Per-row conversion cost.** The row-to-Arrow path did redundant per-row work (repeated map normalization and re-parsing of each response page). This was reduced to a single parse per page and a single-pass row conversion, which is the main reason the client-CPU figures in S1/S2 are as low as they are.

Result completeness

A family of cases silently returned incomplete results and were corrected:

- `GROUP BY` without `LIMIT` returned only the first ~10 groups (the default aggregation size).
- Window `PARTITION BY` truncated to the first ~10 partitions.
- `SELECT` without `LIMIT` on the non-scroll path returned only the first ~10 rows.
- An explicit `LIMIT` above `index.max_result_window` failed while the same query with no `LIMIT` succeeded.

Each now returns the complete result set, and each is covered by a multi-shard completeness test. These matter for a benchmark because a silently truncated result would otherwise look like a fast, correct run.

Result shape

Extraction rows previously carried Elasticsearch hit-metadata columns (`_id`, `_index`, `_score`) in addition to the selected columns. Rows now contain exactly the columns the SQL projects, so the client receives — and pays for — only the requested data (8 columns for the S1 query).

Environment note

The published sidecar image originally shipped without a logging configuration, which caused it to log at DEBUG and write every Elasticsearch response byte to the container log — a large performance and data-exposure problem. The image now ships a logging configuration that keeps wire-level logging off by default; the benchmark runs the sidecar as shipped.